

CI/CD & Test-Harness Engineering

Seminar: Kiểm Thử Phần Mềm Tự Động và Tích Hợp Liên Tục

1. MỞ ĐẦU

1.1 Vấn Đề: Tại Sao Kiểm Thử Thủ Công và Tích Hợp Muộn Gây Lỗi?

Trong phát triển phần mềm truyền thống, quy trình kiểm thử thường gặp phải những vấn đề sau:

Kiểm thử thủ công (Manual Testing):

- Tester phải chạy lại các test case mỗi khi có thay đổi code → tốn thời gian, công sức
- Con người dễ bỏ sót các trường hợp test
- Khó lặp lại test một cách chính xác (kết quả có thể khác nhau)
- Không thể phát hiện regression (lỗi phát sinh từ code cũ bị ảnh hưởng)
- Feedback chậm → developer phải chờ lâu mới biết code có vấn đề

Tích hợp code muộn (Late Integration):

- Lập trình viên làm việc độc lập, không tích hợp code thường xuyên
- Khi tích hợp, phát hiện xung đột code, lỗi giao tiếp giữa module → khó fix
- Lỗi không được phát hiện sớm, chi phí sửa chữa cao
- Thời gian release bị kéo dài

Hệ quả:

- Chất lượng phần mềm không được đảm bảo
- Sản phẩm chứa nhiều bug khi ship tới khách hàng
- Chi phí maintenance và fix bug sau khi release rất cao

1.2 Tại Sao CI/CD Quan Trọng trong Kiểm Thử Phần Mềm?

CI/CD (Continuous Integration / Continuous Delivery/Deployment) giải quyết những vấn đề trên bằng cách:

- **Tích hợp code thường xuyên:** Mỗi lần push code lên repository, hệ thống tự động kiểm thử, phát hiện lỗi ngay lập tức

- **Kiểm thử tự động:** Test chạy nhanh, có thể chạy 100-1000 test mỗi lần, không cần tester chạy thủ công
- **Feedback nhanh:** Developer nhận phản hồi trong vài phút, có thể sửa lỗi ngay
- **Phát hiện lỗi sớm:** Lỗi được catch ở pipeline, không bao giờ lên production
- **Release an toàn và nhanh:** Có thể deploy nhiều lần một ngày mà vẫn đảm bảo chất lượng

1.3 Mục Tiêu của Bài Seminar

Bài seminar này nhằm:

1. **Giải thích** khái niệm CI/CD là gì, tại sao cần thiết trong kiểm thử phần mềm
2. **Trình bày** cấu trúc tip của một CI/CD pipeline phục vụ kiểm thử
3. **Giới thiệu** Test-Harness Engineering: các thành phần, công cụ hỗ trợ
4. **Phân loại** các loại test trong pipeline (unit, integration, API, E2E, v.v.)
5. **Khảo sát** các công cụ phổ biến hỗ trợ CI/CD và kiểm thử tự động
6. **Thảo luận** các vấn đề thực tế và cách khắc phục
7. **Đề xuất** best practices để xây dựng pipeline kiểm thử hiệu quả

2. GIẢI THÍCH KHÁI NIỆM NỀN TẢNG

2.1 CI - Continuous Integration (Tích Hợp Liên Tục)

Định nghĩa:

Continuous Integration là quy trình mà các lập trình viên **tích hợp code vào repository chung liên tục** (thường 1-2 lần/ngày), mỗi lần tích hợp được kiểm tra tự động bằng build và test suite.

Quy trình CI:

```
Developer push code → Trigger pipeline → Auto build → Auto test → Report result
```

Mục đích:

- Phát hiện xung đột code, lỗi compile sớm
- Đảm bảo code luôn ở trạng thái có thể chạy được (working state)
- Giảm thiểu overhead của integration testing
- Tăng tốc độ feedback

Ví dụ thực tế:

- Lập trình viên A viết feature X, push lên branch
- GitHub Actions tự động chạy: build → lint → unit test → integration test
- Nếu test fail, A nhận notification ngay, fix ngay trong vài phút
- Không chờ đến phút cuối mới phát hiện lỗi

2.2 CD - Continuous Delivery / Continuous Deployment

Continuous Delivery (Phân Phối Liên Tục):

- Là tiếp nối của CI: code **sẵn sàng** để deploy tới production bất cứ lúc nào
- Có thể tự động deploy tới staging environment để kiểm thử thêm
- Nhưng **deploy tới production cần manual approval** từ manager/product owner
- Giảm risk vì có lần review cuối cùng trước khi lên production

Continuous Deployment (Triển Khai Liên Tục):

- Là bước tiếp theo: code **tự động deploy tới production** khi vượt qua tất cả test
- Không cần approval thủ công
- Cho phép release rất nhanh, liên tục (nhiều lần/ngày)
- Yêu cầu quality assurance rất cao, test cảnh báo phải tuyệt đối tin cậy

So sánh:

Khía Cạnh	Continuous Integration	Continuous Delivery	Continuous Deployment
Scope	Tích hợp code, chạy test	CD + Deploy staging	CD + Deploy production
Deploy production	Không	Manual approval	Tự động
Tần suất release	Vài lần/ngày	Hàng ngày/tuần	Hàng giờ/ngày
Risk	Thấp (chỉ test)	Trung bình	Cao (cần test tuyệt đối)
Yêu cầu test	Solid	Comprehensive	Exhaustive + monitoring
Phù hợp cho	Mọi dự án	Dự án ổn định	Startup, web, SaaS

2.3 Pipeline (Đường Ống)

Định nghĩa:

Pipeline là một **chuỗi bước (stage) tự động** được thực thi lần lượt, mỗi bước hoàn thành mới tiếp tục bước tiếp theo. Nếu bước nào fail, pipeline dừng lại.

Cấu trúc típ:

```
Trigger (push code)
  ↓
Stage 1: Install dependencies
  ↓
Stage 2: Lint + format check
  ↓
Stage 3: Unit test
  ↓
Stage 4: Build
  ↓
Stage 5: Integration test
  ↓
Stage 6: Security scan
  ↓
Stage 7: Deploy staging
  ↓
Stage 8: E2E test trên staging
  ↓
Stage 9: Manual approval
  ↓
Stage 10: Deploy production
  ↓
Success ✓
```

Lợi ích:

- Tự động hóa quy trình: không cần chạy thủ công
- Fail fast: phát hiện lỗi sớm, không lãng phí resource
- Transparent: mọi người thấy được tiến trình, kết quả

2.4 Vai Trò của Automated Testing trong Pipeline

Automated testing (Kiểm thử tự động) là trái tim của CI/CD:

- **Test chạy nhanh:** 1000 test có thể chạy xong trong vài giây/phút
- **Feedback liền tức:** Developer biết kết quả trong vài phút, không phải chờ
- **Chạy lặp lại:** Có thể chạy lại cùng một test 100 lần, kết quả vẫn giống nhau (repeatability)
- **Bao phủ toàn diện:** Có thể test hàng ngàn scenario mà người không thể
- **Regression detection:** Phát hiện lỗi xảy ra từ code cũ do code mới ảnh hưởng
- **Quality gate:** Đặt yêu cầu test pass, coverage cao mới cho phép merge code

Ví dụ:

- Nếu có 500 unit test, mỗi test 50ms, chạy tuần tự cần 25s
- Chạy song song 4 thread, chỉ cần ~6s
- Có thể chạy sau mỗi commit trong vài giây, không ảnh hưởng developer

3. CÁC THÀNH PHẦN CỦA CI/CD PIPELINE PHỤC VỤ KIỂM THỬ

3.1 Trigger (Bích Kích Hoạt)

Định nghĩa:

Trigger là **sự kiện khởi động pipeline**, là cú chỉ để hệ thống bắt đầu chạy các bước tự động.

Các loại trigger phổ biến:

Trigger	Mô Tả	Khi Nào Dùng	Ví Dụ
Push to branch	Mỗi khi developer push code lên branch (chủ yếu main/develop)	Feature development, mỗi lần commit	Tất cả các dự án
Pull Request (PR)	Mỗi khi tạo hoặc update PR	Kiểm thử trước merge	GitHub PR, GitLab MR
Schedule	Chạy vào thời điểm cố định (e.g., 2h sáng)	Test nightly, test sâu	Nightly build, report
Manual trigger	Developer/manager click nút để chạy	Debug, rerun test	Retry failed test
Release/Tag	Khi tạo git tag phiên bản (v1.0.0)	Build release, deploy production	Release build

Ví dụ cấu hình:

```
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
  schedule:
    - cron: '0 2 * * *' # Chạy 2h sáng mỗi ngày
  workflow_dispatch:      # Manual trigger
```

3.2 Install Dependencies (Cài Đặt Thư Viện Phụ Thuộc)

Mục đích:

- Download và cài đặt toàn bộ dependency (thư viện, package, module) mà project cần
- Đảm bảo environment giống nhau mọi lần chạy

Loại lỗi có thể phát hiện:

- Thiếu dependency khi commit: `package.json` hoặc `go.mod` không được cập nhật
- Version mismatch: lock file không được cập nhật, dùng version sai
- Dependency có lỗi bảo mật (phát hiện nếu kiểm tra)

Tool hỗ trợ:

- **Node.js**: `npm install`, `yarn install`, `pnpm install`
- **Python**: `pip install -r requirements.txt`, `poetry install`
- **Go**: `go mod download`
- **Java**: Maven (`mvn dependency:resolve`), Gradle (`gradle dependencies`)
- **Docker**: `docker pull base image`

Khi nào chạy:

- Mỗi lần pipeline chạy (phần đầu, trước build/test)
- Có thể cache để tăng tốc độ

Best practice:

- Luôn lock version (`package-lock.json`, `poetry.lock`, `go.sum`)
- Cache dependency nếu có thể: GitHub Actions action `actions/setup-node`, `actions/cache`

3.3 Static Analysis / Lint / Format Check

Mục đích:

- Kiểm tra **code style, convention** (indentation, naming, comment)
- Phát hiện **lỗi code tiềm ẩn** (unused variable, unreachable code, type error)
- Kiểm tra **định dạng code** có đúng không
- Không chạy code, chỉ **phân tích source code** (static → không runtime)

Loại lỗi có thể phát hiện:

- Syntax error: typo, missing semicolon (ít gặp trong JS/Python, nhưng gặp nhiều trong Java)
- Logical error: dereference null pointer, unreachable code, dead variable

- Style violation: indentation, naming convention, line length
- Potential bug: shadowed variable, unused import, function never called
- Security issue: hardcoded password, SQL injection risk (nếu tool hỗ trợ)

Tool hỗ trợ:

Tool	Ngôn Ngữ	Loại Lỗi	Ưu Điểm	Hạn Chế
ESLint	JavaScript/TypeScript	Style, logic, security	Cấu hình linh hoạt, plugin đa dạng	Chậm với project lớn
Prettier	JS/TS/JSON/CSS	Format	Tự động fix format, consistent	Chỉ check format, không logic
Pylint / Ruff	Python	Style, logic, security	Strict, nhiều rule	Ruff nhanh hơn nhưng option ít hơn
golangci-lint	Go	Style, logic, security	Tích hợp nhiều linter, nhanh	Cấu hình phức tạp
Checkstyle / PMD	Java	Style, logic	Widely adopted	Config XML dài dòng
SonarQube	Đa ngôn ngữ	Toàn diện: style, logic, security, coverage	Báo cáo chi tiết, web UI	Cần server, setup phức tạp

Khi nào chạy:

- Sớm trong pipeline, ngay sau install dependencies
- Chạy trên mỗi PR, mỗi push
- Fail fast: nếu lint fail, không cần build/test tiếp

Best practice:

- Tích hợp linter vào IDE: developer thấy lỗi ngay khi code
- Auto-fix nếu có thể: ESLint --fix, Prettier --write, golangci-lint fix
- Strict rule: không vì "tiện" mà disable rule

3.4 Build

Mục đích:

- **Biên dịch (compile)** source code thành executable code (Java, Go, C++ → binary)

- **Bundle/minify** code (JavaScript, CSS → production bundle)
- **Type checking** (TypeScript, statically typed language)
- Đảm bảo code có thể chạy được trước test

Loại lỗi có thể phát hiện:

- Compile error: syntax lỗi, type mismatch, missing import
- Build error: dependency issue, version conflict
- Bundling error: circular dependency, missing asset

Tool hỗ trợ:

- **Java**: Maven (mvn clean package), Gradle (gradle build)
- **Go**: go build, golangci-lint build
- **Node.js**: webpack, esbuild, Vite, Parcel
- **Python**: setuptools (python setup.py build), poetry build
- **Docker**: docker build (dùng Dockerfile)

Khi nào chạy:

- Sau lint pass
- Trước test (nếu test cần executable)
- Có thể skip nếu là interpreted language (Python, JS) và không cần bundle

Best practice:

- Parallelize build nếu có thể: Maven parallel build, Go package compilation
- Cache build output: giảm thời gian build lần tới
- Fail on warning: `-Werror` trong compiler, hoặc config tool strict

3.5 Unit Test

Mục đích:

- Test **từng hàm/method riêng lẻ** (smallest testable unit)
- Isolate dependency: dùng mock, stub để tách rời external dependency
- Chạy **nhANH** (mỗi test <100ms)
- Bao phủ **logic/control flow**: decision, loop, exception

Loại lỗi có thể phát hiện:

- Logic error: function output không đúng expected
- Edge case: boundary value, null input, empty list

- Exception handling: function không handle exception properly
- Regression: function đã từng test pass, giờ fail (vì code cũ thay đổi)

Tool hỗ trợ:

- **JavaScript:** Jest, Vitest, Mocha + Chai
- **Python:** pytest, unittest
- **Java:** JUnit (JUnit 4, JUnit 5), TestNG
- **Go:** testing (built-in)
- **C#/.NET:** NUnit, xUnit.NET, MSTest

Khi nào chạy:

- Mỗi lần push, mỗi PR
- Phải pass trước merge
- Nên chạy sớm (ngay sau build) vì nhanh

Best practice:

- Mỗi test test **1 behavior**: 1 assert hoặc 1 expect
- Test name rõ: `testAddTwoPositiveNumbersShouldReturnSum()` chứ không phải `test1()`
- Arrange-Act-Assert: `given(data) → when(action) → then(assert)`
- 80% code coverage (nhưng coverage cao không = chất lượng tốt)

3.6 Integration Test

Mục đích:

- Test **tương tác giữa các module/component** với nhau
- Test **database interaction**: insert, query, update, delete
- Test **API endpoint**: request-response flow
- Test **internal service call**: service A gọi service B

Loại lỗi có thể phát hiện:

- Interface mismatch: module A trả kết quả format X, module B kỳ vọng format Y
- Database issue: query sai, migration chưa chạy, constraint violation
- Transaction issue: rollback, deadlock
- Async issue: race condition, timeout

Tool hỗ trợ:

- **Database:** Testcontainers (Docker container database tạm thời), embedded H2/SQLite
- **API mock:** WireMock, MockServer, Prism
- **Framework:** JUnit + Spring Boot Test (Java), pytest + SQLAlchemy (Python), Go test
- **Test data:** fixtures, factory, seed scripts

Khi nào chạy:

- Sau unit test pass
- Có thể ở PR pipeline (nếu nhanh) hoặc nightly (nếu chậm)
- Phải có test environment (database, external service mock)

Best practice:

- Dùng test database, không dùng production database
- Clean up data sau test: transaction rollback, DELETE
- Mock external service: không call thực service trong test
- Test data setup rõ ràng: fixture hoặc factory method

3.7 API Test

Mục đích:

- Test **REST API endpoint** end-to-end
- Validate **request-response**: HTTP status, headers, body format
- Test **API behavior**: create, read, update, delete (CRUD)
- Test **error case**: invalid input → 400, unauthorized → 401, not found → 404

Loại lỗi có thể phát hiện:

- Wrong status code: endpoint trả 200 khi phải trả 404
- Response schema invalid: field thiếu, type sai
- Business logic error: create user nhưng duplicate ID
- API contract break: API response format thay đổi, client break

Tool hỗ trợ:

- **Postman + Newman**: Test thủ công rồi export, Newman chạy trong CLI
- **REST Assured** (Java): Fluent API, easy assertion
- **Supertest** (Node.js): HTTP assertion library
- **Requests** (Python): Simple HTTP library + pytest
- **Go httptest**: Built-in testing package
- **Cucumber + HTTP library**: BDD style

Khi nào chạy:

- Sau integration test
- Trên staging/production-like environment
- Nên chạy song song (request I/O bound) để tiết kiệm thời gian

Best practice:

- Parametrize test: 1 test function, nhiều input data
- Clear API contract: OpenAPI spec, validate response vs spec
- Don't share test state: mỗi test độc lập, không phụ thuộc test khác
- Test both happy path và error path

3.8 Coverage Report

Mục đích:

- **Đo lường** phần trăm code được test cover
- **Identify** dead code, untested area
- **Trend tracking**: coverage tăng hay giảm theo thời gian
- **Quality gate**: yêu cầu coverage >80% mới merge

Loại lỗi có thể phát hiện:

- Dead code: code write nhưng không ai gọi (hàng trăm line)
- Insufficient test: coverage cao nhưng test edge case ít
- Complex code không test đủ: cyclomatic complexity cao nhưng coverage thấp

Tool hỗ trợ:

- **Java**: JaCoCo (measure code coverage, XML report)
- **JavaScript**: Istanbul / nyc (JavaScript code coverage)
- **Python**: coverage.py (measure, report)
- **Go**: go test -cover, go tool cover
- **Upload to SaaS**: Codecov, Coveralls (track history, CI/CD integration)

Khi nào chạy:

- Mỗi lần unit test / integration test
- Tạo report dưới dạng HTML, XML, hoặc JSON
- Upload lên Codecov / Coveralls để track history, PR comment

Best practice:

- Coverage **40-80%** tùy complexity: không cần 100%
- Focus trên critical path, business logic: coverage 80%+
- Coverage tool không đủ: cần manual review test quality
- Trend: coverage nên tăng hoặc ổn định, không giảm

3.9 Security Scan

Mục đích:

- Scan **dependency** có vulnerability: outdated library, CVE (Common Vulnerabilities and Exposures)
- Scan **source code**: hardcoded password, SQL injection, XSS, insecure API call
- Scan **Docker image**: malware, bad package
- Scan **container registry**: before push/pull

Loại lỗi có thể phát hiện:

- Dependency vulnerability: library version có lỗi bảo mật known
- Code security issue: `String password = "admin123";` hardcoded, SQL injection risk
- Container issue: base image outdated, suspicious package
- License issue: GPL library trong proprietary product

Tool hỗ trợ:

Tool	Loại Scan	Ưu Điểm	Hạn Chế
Snyk	Dependency, code, container	Cloud, UI tốt, frequent update	Freemium, paid plan
OWASP Dependency-Check	Dependency, CVE database	Open source, free, offline mode	Cần maintenance, false positive
Trivy	Dependency, container image	Fast, offline, minimal setup	Ít feature advanced
Semgrep	Source code, pattern matching	Accurate, custom rules, open source	Requires setup, slower
SonarQube	Dependency, code quality, security	Comprehensive, centralized	Expensive, setup phức tạp
GitHub Dependabot	Dependency version update	Auto PR, free với GitHub	GitHub only

Khi nào chạy:

- Sau build, trước test (fail fast nếu critical vuln)
- Hoặc sau test (để không delay feedback)
- Nightly scan hoặc mỗi PR scan (tùy tool)

Best practice:

- Fix critical/high severity issue: CVSS score ≥ 7.0
- Medium/low: prioritize, monitor, không cần immediate fix
- Auto-update dependency: enable Dependabot / Renovate để auto-create PR
- Whitelist false positive: tool có false positive, cần whitelist

3.10 Package Artifact

Mục đích:

- Tạo **binary/jar/image** từ build output
- Lưu artifact vào **artifact storage**: JFrog Artifactory, Docker Registry, GitHub Container Registry
- Version artifact: tag version số, git commit hash
- Cho phép **downstream stage** (deploy, smoke test) dùng lại artifact

Loại lỗi có thể phát hiện:

- Build artifact missing: build output bị xóa hoặc không được tạo
- Wrong version artifact: artifact tag sai version, deploy sai phiên bản
- Artifact corrupted: file bị damage khi upload/download

Tool hỗ trợ:

- **Docker Image**: docker build → docker push (Docker Hub, GCR, ECR, ACR)
- **JAR File**: Maven Central, JFrog Artifactory
- **NPM Package**: npm publish (npmjs.com, private registry)
- **GitHub Release**: github-cli (gh release create)
- **Artifact Storage**: GitHub Actions artifacts, AWS S3, Azure Blob Storage

Khi nào chạy:

- Sau test pass, coverage check pass
- Trước deploy
- Thường ở lúc tạo release/tag hoặc merge vào main

Best practice:

- Tag artifact với: `<app>:<version>-<commit-hash>` e.g., `myapp:1.0.0-abc123f`
- Keep artifact tối thiểu: chỉ lưu production artifact, không lưu debug/test artifact
- Set retention policy: xóa artifact cũ sau N ngày để tiết kiệm storage

3.11 Deploy Staging

Mục đích:

- Deploy artifact tới **staging environment** (giống production nhưng là test)
- Prepare để **test end-to-end**, smoke test
- Validate deployment process: scripts, config, rollback

Loại lỗi có thể phát hiện:

- Deployment script error: config sai, environment variable missing
- Missing dependency: staging không có required service
- Version conflict: staging cũ, artifact mới không compatible
- Database migration fail: schema change không apply

Tool hỗ trợ:

- **Docker Compose**: docker-compose up (local staging)
- **Kubernetes**: kubectl apply, helm install (cloud staging)
- **Cloud Deploy**: AWS CodeDeploy, Azure Pipelines, GCP Cloud Deploy
- **SSH + script**: ansible, terraform (infrastructure as code)

Khi nào chạy:

- Sau artifact ready
- Trước smoke test / E2E test
- Có thể ở nightly pipeline để không delay PR feedback

Best practice:

- Staging configuration **giống production**: OS, database, dependencies version
- Infrastructure as Code: Terraform, CloudFormation, Kubernetes yaml
- Automated rollback: nếu health check fail, rollback version cũ
- Smoke test sau deploy: quick sanity check trước E2E

3.12 Smoke Test / E2E Test

Mục đích:

- **Smoke test:** Quick sanity check, basic functionality works
 - E.g., homepage load, login endpoint response, database connected
 - Mục đích: detect obvious issues nhanh (chỉ vài phút)
- **E2E test:** Full user journey test (end-to-end)
 - E.g., user login → view list → create item → logout
 - Validate real-world workflow, integration

Loại lỗi có thể phát hiện:

- Deployment failed: service không start, port not listening
- Config issue: environment variable sai, database connection fail
- Integration fail: frontend call backend sai URL
- UI bug: button không click, form validation error
- Real-world scenario: user can't login, can't create resource

Tool hỗ trợ:

- **UI Test:** Playwright, Cypress, Selenium
- **API Test:** Postman Newman, REST Assured, Supertest
- **Load Test (performance):** k6, JMeter, Gatling
- **Headless:** Can run browser automated

Khi nào chạy:

- Sau deploy staging
- Thường ở release pipeline, nightly pipeline (chậm, expensive)
- Có thể run on-demand (manual trigger)

Best practice:

- Test critical path: login, purchase, payment
- Use test data: separate test account, test credit card number
- Timeout handling: web slow, retry, don't fail immediately
- Parallel test: run browser parallel (multiple users) để tiết kiệm thời gian

3.13 Manual Approval / Production Deploy

Mục đích:

- **Manual Approval:** Human review trước khi deploy production
 - Manager/product owner check: feature OK? bug fixed? ready ship?
 - Risk assessment: có breaking change? liên hệ customer?

- **Production Deploy:** Tự động deploy approved code tới production
 - Sử dụng blue-green deployment hoặc canary để reduce risk
 - Health check sau deploy

Loại lỗi có thể phát hiện:

- Human error: feature not meant to release yet, incomplete fix
- Breaking change: API change not communicated to client
- Data issue: database migration destroy data
- Downtime: deployment cause outage (avoided by blue-green/canary)

Tool hỗ trợ:

- **Pipeline approval:** GitHub environment, GitLab approval rule, Jenkins approval step
- **Deployment:** Kubernetes kubectl, Helm, Ansible, Terraform
- **Blue-green:** Traffic switch từ blue (old) → green (new) instantly
- **Canary:** Route 5% traffic → new version, monitor, gradually increase
- **Health check:** HTTP /health endpoint, database query, external service ping

Khi nào chạy:

- Cuối cùng, sau tất cả test pass
- Có gate: require approval từ designated person
- Release thường theo schedule: mỗi tuần, mỗi 2 tuần

Best practice:

- Approval checklist: feature demo, test pass, documentation ready
- Notification: Slack/email notification tới team trước/sau deploy
- Monitoring alert: deploy done → alert service health, error rate
- Rollback plan: nếu production có issue, có kế hoạch rollback

4. TEST-HARNESS ENGINEERING

4.1 Test Harness Là Gì?

Định nghĩa:

Test Harness (Bộ Công Cụ Kiểm Thử) là **tập hợp các thành phần và công cụ** được xây dựng để **hỗ trợ chạy test một cách tự động, lặp lại, đáng tin cậy**.

Analogies:

- Giống như **scaffolding** (giàn giáo) khi xây nhà: hỗ trợ build, nhưng không phải build chính
- Test harness là **infrastructure** để test, không phải test code chính

Thành phần:

1. Test runner: Công cụ chạy test
2. Test script: Code test
3. Test data: Dữ liệu để test
4. Fixture: Setup/teardown
5. Mock, Stub: Simulate external service
6. Driver: Control System Under Test (SUT)
7. Assertion: Validate kết quả
8. Test environment: Docker, database, config
9. Reporter: Output kết quả
10. Logging/artifact: Lưu log, screenshot, trace

Ví dụ:

```
Test Harness =  
  Test Runner (Jest)  
  + Test Script (test code)  
  + Test Data (fixtures/factory)  
  + Mock HTTP (WireMock)  
  + Docker Compose (database)  
  + Assertion Library (expect)  
  + Reporter (JUnit XML)
```

4.2 Tại Sao Test Harness Quan Trọng trong Kiểm Thử Tự Động?

1. **Consistency**: Mỗi lần test chạy setup giống nhau → kết quả reproducible
2. **Speed**: Không phải setup thủ công, chạy nhanh
3. **Scalability**: Có thể chạy hàng ngàn test mà không phải double-check
4. **Maintainability**: Test data tập trung, dễ update
5. **Reliability**: Mock/stub external service → test không phụ thuộc bên ngoài
6. **Reporting**: Test result rõ ràng, dễ debug khi fail

Mà không test harness:

- Setup thủ công: tester phải setup database, start service thủ công
- Chậm: mỗi test cần nhiều thời gian setup
- Không consistent: setup khác nhau → test result khác nhau (flaky)

- Khó maintain: test data scatter đây đó, khó update
- Khó debug: fail mà không rõ nguyên nhân

4.3 Phân Biệt: Test Automation, Test Framework, Test Harness, CI/CD Pipeline

Confusing terms:

Thuật Ngữ	Định Nghĩa	Scope	Ví Dụ
Test Automation	Quy trình tự động hóa kiểm thử (viết test code, chạy tự động)	Quy trình, methodology	"Automate unit test" = viết code test
Test Framework	Thư viện/platform cung cấp API để viết test (assert, mock, runner)	Library	Jest, JUnit, pytest, Cypress
Test Harness	Toàn bộ infrastructure để chạy test (framework + data + mock + env)	System	Docker Compose + Jest + Fixtures
CI/CD Pipeline	Quy trình tự động: build → test → deploy, orchestrate từ git event	Quy trình, automation	GitHub Actions workflow chạy lint → test → deploy

Mối quan hệ:

```

Test Automation (quá trình)
↓
Test Framework (tool): Jest, JUnit
↓
Test Harness (infra): Framework + data + mock + env
↓
CI/CD Pipeline (orchestration): Tự động chạy harness sau mỗi commit

```

4.4 Các Thành Phần của Test Harness

4.4.1 Test Runner

Là gì: Công cụ thực thi test script, report result

Ví dụ:

- Jest (Node.js)
- pytest (Python)

- JUnit (Java)
- Go testing
- Mocha (Node.js)

Chức năng:

- Discover test files (pattern: `*.test.js` , `test_*.py`)
- Execute test code
- Aggregate result (pass/fail)
- Generate report (JSON, XML, HTML)
- Exit code: 0 = pass, non-zero = fail (giúp CI/CD pipeline detect)

4.4.2 Test Script

Là gì: Đoạn code kiểm thử, chứa test case

Ví dụ:

```
// Jest test script
describe('UserService', () => {
  it('should create user with valid email', async () => {
    // Arrange
    const userData = { name: 'John', email: 'john@example.com' };

    // Act
    const result = await userService.create(userData);

    // Assert
    expect(result.id).toBeDefined();
    expect(result.email).toBe('john@example.com');
  });
});
```

Best practice:

- 1 test = 1 behavior
- Name: `testCreateUserWithValidEmailShouldReturnUserId`
- Arrange-Act-Assert pattern
- Avoid multiple assert (1-3 is OK)

4.4.3 Test Data

Là gì: Dữ liệu đầu vào test (input data, expected output)

Cách cung cấp:

1. Hardcoded trong test: OK nếu simple

```
const user = { id: 1, name: 'John', email: 'john@example.com' };
```

2. Fixtures: File JSON/YAML chứa test data

```
// fixtures/user.json
{
  "validUser": { "name": "John", "email": "john@example.com" }
}
```

3. Factory method: Hàm tạo test data

```
function createUser(overrides = {}) {
  return { id: 1, name: 'John', email: 'john@example.com', ...overrides };
}
```

4. Test data builder: Builder pattern

```
new UserBuilder().withName('John').withEmail('john@example.com').build();
```

Best practice:

- Reuse: không lặp test data
- Clear: data tên rõ ràng: `validUser`, `inactiveUser`, `userWithoutEmail`
- Minimal: chỉ include cần thiết, không thêm field không dùng
- Separate: test data ở folder riêng, không mix với test code

4.4.4 Fixture

Là gì: Setup/teardown trước/sau test, chuẩn bị test environment

Ví dụ:

```
// Jest before/after hooks
beforeAll(() => {
  // Setup: start database, create table
  database.connect();
  database.migrate();
});

beforeEach(() => {
  // Setup before mỗi test: clear data
  database.clearUsers();
});
```

```

afterEach(() => {
  // Cleanup sau mỗi test: delete temp file, close connection
  fs.rmSync('/tmp/test', { recursive: true });
});

afterAll(() => {
  // Cleanup: stop database
  database.disconnect();
});

```

Mục đích:

- **Isolation:** Mỗi test độc lập, không bị affect bởi test trước
- **Consistency:** Mỗi test start với state như nhau
- **Efficiency:** Reuse resource (database, server)

4.4.5 Mock

Là gì: Giả lập object/service, track method call, control return value

Ví dụ - Mock HTTP request:

```

const userService = {
  getUser: jest.fn().mockResolvedValue({ id: 1, name: 'John' })
};

// Or dengan WireMock:
// Stub: GET /api/users/1 → { id: 1, name: 'John' }

```

Mục đích:

- Isolate unit: test function A mà không phải call real function B
- Control external service: simulate error (timeout, 500), success case
- Speed: mock faster than real call
- Reliability: real service unreliable → test flaky, mock → stable

Khi nào dùng:

- Unit test: always mock external dependency
- Integration test: mock third-party service (payment, email), real database
- E2E test: real service (hoặc staging copy)

4.4.6 Stub

Là gì: Similar to mock, nhưng chỉ provide fake return value, không track call

Khác nhau Mock vs Stub:

Khía Cạnh	Mock	Stub
Track call	Có (verify nó được gọi)	Không
Return value	Có thể dynamic	Cố định
Assertion	Verify method called with correct args	Verify returned value
Ví dụ	<code>expect(spy).toHaveBeenCalled(...)</code>	<code>stub.returns(100)</code>

Ví dụ stub:

```
// Stub database: luôn trả cùng kết quả
const dbStub = {
  getUser: () => ({ id: 1, name: 'John' })
};
```

4.4.7 Driver

Là gì: Code để kiểm soát System Under Test (SUT), tương tác với component

Ví dụ:

```
// Driver: kiểm soát web page
class LoginPageDriver {
  async login(username, password) {
    await page.fill('#username', username);
    await page.fill('#password', password);
    await page.click('#loginBtn');
    await page.waitForNavigation();
  }
}

// Usage:
const driver = new LoginPageDriver(page);
await driver.login('user@example.com', 'password123');
```

Mục đích:

- Abstraction: hide implementation detail
- Reuse: multiple test dùng cùng driver

- Maintainability: change UI → chỉ update driver, không update tất cả test

4.4.8 Assertion

Là gì: Kiểm tra kết quả có đúng hay không

Ví dụ:

```
expect(result).toBe(5);
expect(user.email).toMatch(/^^[^s@]+@[^s@]+\.[^s@]+$/);
expect(array).toHaveLength(3);
expect(fn).toThrow(Error);
```

Best practice:

- 1-3 assertion per test (focus)
- Clear message: `expect(status).toBe(200, 'Response status should be 200')`
- Use matcher: `toBe`, `toEqual`, `toMatch` instead of `===`

4.4.9 Test Environment

Là gì: Môi trường để test chạy (database, server, network)

Thành phần:

1. **Database:** PostgreSQL, MySQL, MongoDB
 - Setup: migration, seed data
 - Cleanup: transaction rollback, delete data
2. **External service mock:** WireMock, MockServer, LocalStack
 - Mock API, message queue, storage
3. **Docker Compose:** Orchestrate multiple containers

```
version: '3'
services:
  postgres:
    image: postgres:14
    environment:
      POSTGRES_PASSWORD: password
  app:
    build: .
    depends_on:
      - postgres
```

4. **Config/secrets:** Environment variable, secret file

- Avoid hardcode password: dùng env var hoặc .env.test

Best practice:

- Isolated: test environment riêng, không ảnh hưởng production/staging
- Reproducible: Docker/container để consistent environment
- Cleanup: data cleanup sau test, container stop/remove

4.4.10 Reporter

Là gì: Output kết quả test dưới dạng dễ đọc

Format:

- **Console:** Real-time output khi test chạy

```
PASS src/user.test.js
  UserService
    ✓ should create user (45ms)
    ✓ should update user (32ms)
    ✓ should delete user (28ms)
```

```
Test Suites: 1 passed, 1 total
Tests: 3 passed, 3 total
```

- **JUnit XML:** Machine-readable, CI/CD tools parse

```
<testsuite name="UserService" tests="3" failures="0">
  <testcase name="should create user" time="0.045"/>
</testsuite>
```

- **HTML:** Browser-friendly report

```
<h1>Test Report</h1>
<table>
  <tr><td>should create user</td><td>✓ PASS</td></tr>
</table>
```

- **JSON:** Custom parsing

```
{ "tests": 3, "failures": 0, "duration": 150 }
```

Tool:

- Jest: default console, optional coverage report
- pytest: `-v` verbose, `--html` HTML report
- JUnit: XML native

- Allure: beautiful HTML report

4.4.11 Log / Artifact

Là gì: Chi tiết debug khi test fail

Loại artifact:

- **Log file:** `test.log` containing stdout/stderr
- **Screenshot:** UI test fail → screenshot HTML page
- **Video:** Cypress/Playwright record video
- **Trace:** Browser DevTools trace
- **Database dump:** Export database state khi test fail
- **Network log:** Request/response log

Mục đích:

- Debug: khi test fail, log/screenshot giúp understand nguyên nhân
- Investigation: rerun test locally dùng artifact
- Documentation: test evidence

Best practice:

- Capture fail only: không lưu artifact thành công (tiết kiệm storage)
- Structured: folder theo test name, date
- Accessible: store ở artifact storage (S3, GitHub artifacts)

5. VÍ DỤ TEST HARNESS: API Backend + Database

Dưới đây là ví dụ **đầy đủ test harness** cho project Node.js backend:

5.1 Cấu Trúc Project

```
project/
├── src/
│   ├── user.service.js      # Business logic
│   ├── user.controller.js   # API endpoint
│   └── database.js          # DB connection
├── test/
│   ├── fixtures/
│   │   └── user.json        # Test data
│   └── mocks/
```

```

|   |   └─ emailService.mock.js    # Mock service
|   └─ setup.js                    # Global setup
|   └─ user.service.test.js        # Test code
└─ docker-compose.test.yml         # Test environment
└─ jest.config.js                  # Test runner config
└─ package.json

```

5.2 Test Environment - docker-compose.test.yml

```

version: '3.8'
services:
  postgres:
    image: postgres:14-alpine
    environment:
      POSTGRES_USER: testuser
      POSTGRES_PASSWORD: testpass
      POSTGRES_DB: testdb
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U testuser"]
      interval: 5s
      timeout: 5s
      retries: 5

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

```

5.3 Global Setup - test/setup.js

```

// test/setup.js
const { Pool } = require('pg');
const path = require('path');
require('dotenv').config({ path: path.resolve(__dirname, '../.env.test') });

let pool;

beforeAll(async () => {
  // 1. Connect to test database
  pool = new Pool({
    user: process.env.DB_USER || 'testuser',
    password: process.env.DB_PASSWORD || 'testpass',

```

```

    host: process.env.DB_HOST || 'localhost',
    database: process.env.DB_NAME || 'testdb'
  });

  // 2. Run migration
  await runMigration(pool);

  // 3. Seed initial data if needed
  await seedTestData(pool);
});

afterAll(async () => {
  // Cleanup: close database connection
  await pool.end();
});

// Helper: run migration
async function runMigration(pool) {
  const client = await pool.connect();
  try {
    await client.query(`
      CREATE TABLE IF NOT EXISTS users (
        id SERIAL PRIMARY KEY,
        name VARCHAR(255) NOT NULL,
        email VARCHAR(255) UNIQUE NOT NULL,
        created_at TIMESTAMP DEFAULT NOW()
      );
    `);
  } finally {
    client.release();
  }
}

// Helper: seed data
async function seedTestData(pool) {
  const client = await pool.connect();
  try {
    await client.query('DELETE FROM users;');
    await client.query(`
      INSERT INTO users (name, email) VALUES
      ('Admin User', 'admin@example.com'),
      ('Test User', 'test@example.com');
    `);
  } finally {
    client.release();
  }
}

```

```
}

// Export for test access
module.exports = { pool };
```

5.4 Test Fixtures - test/fixtures/user.json

```
{
  "validUser": {
    "name": "John Doe",
    "email": "john.doe@example.com"
  },
  "userWithoutEmail": {
    "name": "Jane Doe"
  },
  "userWithInvalidEmail": {
    "name": "Bob",
    "email": "invalid-email"
  },
  "existingUser": {
    "id": 1,
    "name": "Admin User",
    "email": "admin@example.com"
  }
}
```

5.5 Mock External Service - test/mocks/emailService.mock.js

```
// test/mocks/emailService.mock.js
class EmailServiceMock {
  constructor() {
    this.sentEmails = [];
  }

  async send(to, subject, body) {
    // Simulate email send (just store in memory)
    this.sentEmails.push({ to, subject, body, timestamp: new Date() });
    return { success: true, messageId: `mock-${Date.now()}` };
  }

  getSentEmails() {
    return this.sentEmails;
  }
}
```

```

    clear() {
      this.sentEmails = [];
    }
  }
}

module.exports = new EmailServiceMock();

```

5.6 Test Case - test/user.service.test.js

```

// test/user.service.test.js
const UserService = require('../src/user.service');
const { pool } = require('./setup');
const emailServiceMock = require('./mocks/emailService.mock');
const fixtures = require('./fixtures/user.json');

describe('UserService', () => {
  let userService;

  beforeEach(() => {
    // Setup: create service with mocked email
    userService = new UserService(pool, emailServiceMock);

    // Clear mock call history
    emailServiceMock.clear();
  });

  describe('createUser', () => {
    it('should create user with valid email and send welcome email', async ()
=> {
      // Arrange
      const userData = fixtures.validUser;

      // Act
      const result = await userService.create(userData);

      // Assert - User created
      expect(result.id).toBeDefined();
      expect(result.name).toBe(fixtures.validUser.name);
      expect(result.email).toBe(fixtures.validUser.email);

      // Assert - Welcome email sent
      const sentEmails = emailServiceMock.getSentEmails();
      expect(sentEmails).toHaveLength(1);
      expect(sentEmails[0].to).toBe(fixtures.validUser.email);
      expect(sentEmails[0].subject).toContain('Welcome');
    });
  });
});

```

```

});

it('should throw error when email already exists', async () => {
  // Arrange: existing user already in database
  const userData = fixtures.existingUser;

  // Act & Assert
  await expect(userService.create(userData))
    .rejects
    .toThrow('Email already exists');
});

it('should throw error when email invalid', async () => {
  // Arrange
  const userData = fixtures.userWithInvalidEmail;

  // Act & Assert
  await expect(userService.create(userData))
    .rejects
    .toThrow('Invalid email format');
});
});

describe('getUser', () => {
  it('should return user by ID', async () => {
    // Arrange
    const userId = 1;

    // Act
    const result = await userService.getById(userId);

    // Assert
    expect(result).toBeDefined();
    expect(result.id).toBe(userId);
    expect(result.email).toBe(fixtures.existingUser.email);
  });

  it('should return null when user not found', async () => {
    // Arrange
    const userId = 99999;

    // Act
    const result = await userService.getById(userId);

    // Assert
    expect(result).toBeNull();
  });
});

```

```

    });
  });

  describe('updateUser', () => {
    it('should update user name', async () => {
      // Arrange
      const userId = 1;
      const newName = 'Updated Admin';

      // Act
      const result = await userService.update(userId, { name: newName });

      // Assert
      expect(result.name).toBe(newName);

      // Verify in database
      const dbUser = await userService.getById(userId);
      expect(dbUser.name).toBe(newName);
    });
  });

  describe('deleteUser', () => {
    it('should delete user', async () => {
      // Arrange
      const userId = 1;

      // Act
      await userService.delete(userId);

      // Assert
      const result = await userService.getById(userId);
      expect(result).toBeNull();
    });
  });
});

```

5.7 Jest Configuration - jest.config.js

```

// jest.config.js
module.exports = {
  testEnvironment: 'node',
  testMatch: ['<rootDir>/test/**/*.test.js'],
  setupFilesAfterEnv: ['<rootDir>/test/setup.js'],
  testTimeout: 10000, // 10 second timeout per test
  collectCoverageFrom: [

```

```

    'src/**/*.js',
    '!src/index.js'
  ],
  coverageThreshold: {
    global: {
      branches: 80,
      functions: 80,
      lines: 80,
      statements: 80
    }
  }
};

```

5.8 Run Test Harness

```

# Start test environment (docker)
docker-compose -f docker-compose.test.yml up -d

# Run test
npm test

# Generate coverage report
npm test -- --coverage

# Clean up
docker-compose -f docker-compose.test.yml down

```

Test output:

```

PASS test/user.service.test.js (2.5s)
  UserService
    createUser
      ✓ should create user with valid email and send welcome email (45ms)
      ✓ should throw error when email already exists (28ms)
      ✓ should throw error when email invalid (32ms)
    getUser
      ✓ should return user by ID (15ms)
      ✓ should return null when user not found (10ms)
    updateUser
      ✓ should update user name (25ms)
    deleteUser
      ✓ should delete user (20ms)

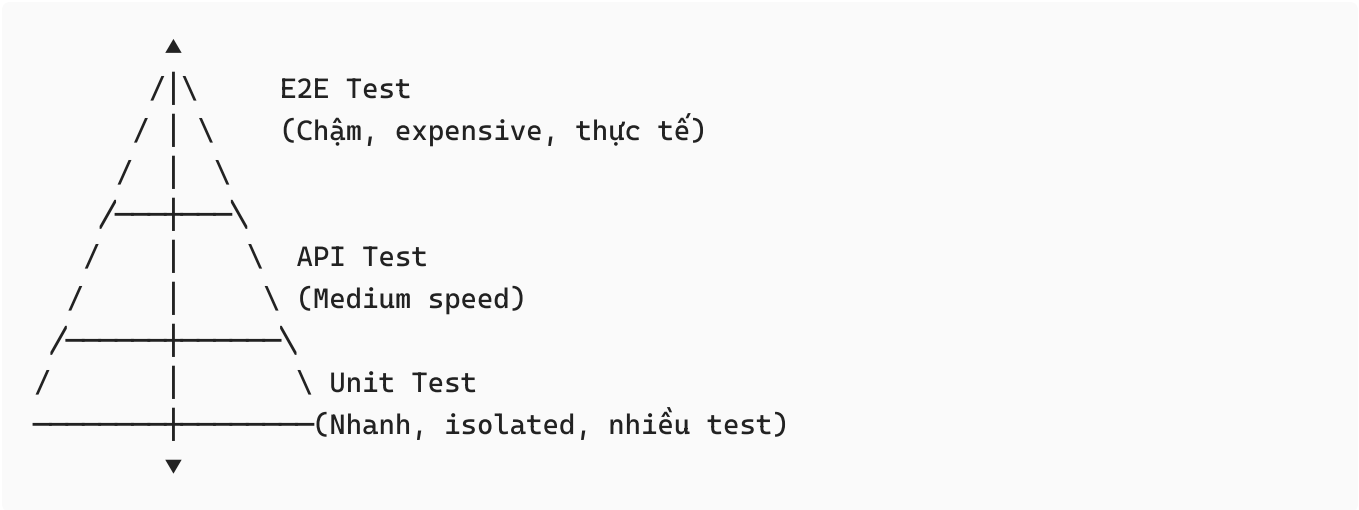
Test Suites: 1 passed, 1 total
Tests:       7 passed, 7 total

```


Snapshots: 0 total
Time: 2.542 s
Coverage: Lines 85%, Branches 80%, Functions 90%

6. CÁC LOẠI TEST TRONG CI/CD

6.1 Test Pyramid



6.2 Unit Test

Khía Cạnh	Chi Tiết
Mục tiêu	Test 1 function/method riêng lẻ, không gọi external dependency
Vị trí pipeline	Sớm, sau build (trước all other test)
Speed	Rất nhanh: 10-100ms per test, 100 test trong <5s
Ưu điểm	• Nhanh → feedback ngay • Isolate → easy debug • Cheap → run mỗi commit • Bao phủ edge case tốt
Hạn chế	• Không test integration • Mock quá nhiều → unrealistic • Coverage cao không = quality cao
Tool	Jest (JS), pytest (Python), JUnit (Java), Go testing
Ví dụ	Test <code>add(2, 3)</code> return 5, test <code>isEmail()</code> validate format
Coverage goal	>80%, ideally ~85%

6.3 Integration Test

Khía Cạnh	Chi Tiết
Mục tiêu	Test interaction giữa modules: service A + database, service A + service B
Vị trí pipeline	Sau unit test
Speed	Medium: 100ms-1s per test, 50 test trong 30-50s
Ưu điểm	• Test real interaction • Database/service real • Catch integration bug
Hạn chế	• Chậm hơn unit test • Dependency trên external (database, service) → flaky risk • Harder to debug
Tool	Testcontainers (Docker DB), WireMock (mock service), Spring Test (Java), pytest
Ví dụ	Test UserService.create() + database insert, OrderService.checkout() + PaymentService call
Setup	Docker Compose, seed data, cleanup

6.4 API Test

Khía Cạnh	Chi Tiết
Mục tiêu	Test REST API endpoint end-to-end: request → response
Vị trí pipeline	Sau integration test, hoặc ở staging
Speed	Medium: 100ms-500ms per request, 50 test trong 10-30s (network latency)
Ưu điểm	• Test real API • Validate schema/status • Easy parametrize
Hạn chế	• Chậm (network I/O) • Dependent on server running • State sharing (test A create, test B update)
Tool	Postman/Newman, Supertest (JS), REST Assured (Java), httpx (Python)
Ví dụ	POST /users (201), GET /users/1 (200), DELETE /users/1 (204)
Best practice	Parametrize, independent test, clear assertion

6.5 End-to-End (E2E) Test

Khía Cạnh	Chi Tiết
Mục tiêu	Test full user journey: login → navigate → action → logout

Khía Cạnh	Chi Tiết
Vị trí pipeline	Release pipeline, staging environment, nightly
Speed	Chậm: 1-5s per test, 20 test trong 1-2 phút
Ưu điểm	• Test real user flow • Catch UI bug • Real browser (JS execution, CSS rendering)
Hạn chế	• Rất chậm → không chạy mỗi PR • Flaky: timing issue, element not found • Expensive: browser resource
Tool	Playwright, Cypress, Selenium, Puppeteer
Ví dụ	User login → view product list → add to cart → checkout → payment → confirm order
Best practice	Test critical path only, use test data, handle timeout, parallel execution

6.6 Regression Test

Khía Cạnh	Chi Tiết
Mục tiêu	Đảm bảo code mới không break feature cũ
Vị trí pipeline	Sau unit/integration/API test
Speed	Tùy loại test: unit (nhanh), E2E (chậm)
Ưu điểm	• Catch breaking change • Prevent known bug reappear
Hạn chế	• Có thể overlap với unit/integration test
Tool	Không tool riêng, là cách sử dụng test hợp lý
Ví dụ	Sau fix bug login, chạy toàn bộ test để đảm bảo login còn work
Best practice	Maintain regression test suite, run regularly

6.7 Smoke Test

Khía Cạnh	Chi Tiết
Mục tiêu	Quick sanity check sau deploy: application start? database connected?
Vị trí pipeline	Sau deploy staging/production
Speed	Rất nhanh: 10-30s total
Ưu điểm	• Detect obvious deployment error nhanh • Reduce wasted resource (nếu fail, stop ở đây)

Khía Cạnh	Chi Tiết
Hạn chế	• Không comprehensive, chỉ basic check
Tool	HTTP healthcheck, ping endpoint, simple curl
Ví dụ	GET / (200), GET /api/health (200), Database query (connected)
Best practice	Minimal test, fast execution, clear failure message

6.8 Performance / Load Test

Khía Cạnh	Chi Tiết
Mục tiêu	Test khi load cao: response time, throughput, memory usage
Vị trí pipeline	Nightly, release pipeline (optional)
Speed	Chậm: 5-30 phút (simulate many users)
Ưu điểm	• Find bottleneck • Baseline performance • SLA validation
Hạn chế	• Chậm → không chạy mỗi PR • Costly (resource intensive) • Environment-dependent
Tool	k6, JMeter, Gatling, wrk
Ví dụ	Simulate 1000 concurrent user, verify response time <500ms, throughput >10k req/s
Best practice	Realistic load profile, clear metric, alerting on regression

6.9 Security Test

Khía Cạnh	Chi Tiết
Mục tiêu	Phát hiện vulnerability: SQL injection, XSS, weak password, unauthorized access
Vị trí pipeline	Dependency scan (sớm), code scan (mỗi PR), container scan (pre-push)
Speed	Tùy tool: quick scan (30s), deep scan (5+ phút)
Ưu điểm	• Early vulnerability detection • Prevent known CVE • Automated (không dùng manual penetration test)
Hạn chế	• False positive • Chưa catch business logic flaw (cần manual security review)
Tool	Dependency: Snyk, OWASP DC, Dependabot

Khía Cạnh	Chi Tiết
Ví dụ	Login endpoint require HTTPS, SQL param properly escaped, no hardcoded password
Best practice	Multiple layer scanning, auto-remediation (Dependabot PR), whitelist false positive

7. KHẢO SÁT CÔNG CỤ HỖ TRỢ

7.1 CI/CD Platforms

Platform	Mục Đích	Ưu Điểm	Hạn Chế	Phù Hợp
GitHub Actions	CI/CD platform tích hợp GitHub	• Free với GitHub • Simple YAML • Marketplace action rich • Fast (VMs mới)	• GitHub-only • Matrix build ít flexible • Storage artifact 90 ngày	Dự án GitHub, startup
GitLab CI/CD	CI/CD platform tích hợp GitLab	• Powerful pipeline (DAG) • On-premise option • Shell runner flexible	• GitLab-only • Learning curve • Self-hosted cần infra	Dự án GitLab, enterprise
Jenkins	Self-hosted CI/CD server	• Very flexible • Huge plugin ecosystem • On-premise control	• Setup phức tạp • Maintenance overhead • Not cloud-native	Large enterprise, custom flow
CircleCI	SaaS CI/CD platform	• Fast • Good UX • Config.yml simple • Generous free tier	• Limited workflow complex • Paid plan expensive	Startup, small team
Azure DevOps Pipelines	Azure-integrated CI/CD	• Tích hợp Azure • Enterprise feature • Parallel job abundant	• Pricing confusing • UI complex • Non-Microsoft less ideal	Microsoft stack, enterprise

7.2 Test Framework / Test Runner

Tool	Ngôn Ngữ	Tính Năng	Ưu Điểm	Hạn Chế	Phù Hợp
Jest	JavaScript/TypeScript	Unit, snapshot, coverage	• Fast (parallel) • Zero config • Great DX •	• Chỉ Node.js ecosystem •	Node.js project

Tool	Ngôn Ngữ	Tính Năng	Ưu Điểm	Hạn Chế	Phù Hợp
			Coverage built-in	Snapshot flaky	
pytest	Python	Unit, parametrize, fixture	• Simple API • Powerful fixture • Plugin ecosystem	• Slow (serial default) • config.ini • verbose	Python
Vitest	JavaScript/TypeScript	Unit, ESM-first	• Fast (Vite-powered) • ESM native • Jest-compatible	• Newer, less mature • Smaller community	Modern JS/TypeScript project
JUnit 5	Java	Parameterized, extension model	• Annotation-based • Extension flexible • Modern	• Java verbose • IDE setup needed	Java/Spring project
Go testing	Go	Built-in, benchmark, pprof	• Built-in (no dependency) • Simple • Parallel	• Feature minimal • No assertion library	Go project
NUnit/xUnit	C#/.NET	Parametrized, async	• Modern • .NET integrated	• Windows-centric • Less community	.NET project

7.3 Code Quality / Static Analysis

Tool	Loại	Ưu Điểm	Hạn Chế	Phù Hợp
SonarQube / SonarCloud	Centralized	• Comprehensive (security, quality, coverage) • Web UI nice • Trend Enterprise	• Setup phức tạp • Cost (\$)	Large project, enterprise
ESLint	JavaScript linter	• Flexible config • Plugin rich • Auto-fix	• Slow large project	JavaScript/TypeScript
Prettier	JavaScript formatter	• Opinionated (no config) • Fast	• Not for logic	JavaScript/TypeScript

Tool	Loại	Ưu Điểm	Hạn Chế	Phù Hợp
		Consistent	check	
golangci-lint	Go linter	• Fast • Integrated tools • Parallel	• Config complex	Go project
Pylint / Ruff	Python	• Strict • Many rule	• Ruff fast, pylint feature-rich	Python project
Checkstyle	Java	• Widely used • Customizable	• Config XML verbose	Java

7.4 Coverage Tools

Tool	Ngôn Ngữ	Tính Năng	Ưu Điểm	Hạn Chế	Phù Hợp
JaCoCo	Java	Code coverage, offline instrument	• Fast • Standard • Maven integration	• XML config verbose	Java/Maven
Istanbul / nyc	JavaScript	Coverage reporter	• Simple • JSON output • Multi-reporter	• Deprecated (Jest built-in)	JavaScript/Node.js
coverage.py	Python	Coverage measure	• Simple • Coverage API	• Not AST-aware (can miss edge)	Python
Go coverage	Go	Built-in	• Simple • HTML report	• Feature minimal	Go
Codecov	All	SaaS coverage tracker	• Track history • Badge • GitHub integration	• SaaS cost	CI/CD pipeline

7.5 Environment & Dependency Simulation

Tool	Mục Đích	Ưu Điểm	Hạn Chế	Phù Hợp
Docker	Containerize app	• Reproducible env • Lightweight • Ecosystem	• Overhead vs VM	Every project
Docker Compose	Multi-container orchestration	• Simple YAML • Local dev/test • Dependency order	• Not for production	Local CI/CD test env
Testcontainers	Docker container test	• Programmatic setup • Language native • Cleanup auto	• Java/JVM-centric • Docker required	Integration test
WireMock	Mock HTTP server	• Flexible stubbing • Record/playback • Standalone	• HTTP only (not message queue)	API test, integration
MockServer	Mock HTTP/HTTPS	• Advanced matching • CORS • Multiple format	• Learning curve	Complex API mock
LocalStack	Local AWS simulation	• Offline S3/SQS/DynamoDB • Development • Testing	• Not 100% AWS parity	AWS-based project

7.6 E2E / API / Performance / Security Testing

Tool	Loại	Ưu Điểm	Hạn Chế	Phù Hợp
Playwright	E2E / UI	• Fast • Cross-browser • Good API • Video/trace	• Newer than Cypress	Modern web a
Cypress	E2E / UI	• DX excellent • Easy setup • Real-time debug	• Single browser (Chrome)	Small-medium project
Selenium	E2E / UI	• Mature • Multi-browser • Language support	• Slow • Flaky • Setup complex	Enterprise (mandatory browser)
Postman/Newman	API test	• No-code UI •	• Limited script •	Non-dev API t

Tool	Loại	Ưu Điểm	Hạn Chế	Phù Hợp
		Collection export • Newman CLI	Paid features	
REST Assured	API test	• Fluent API • Java native • BDD-style	• Java-only	Java/Spring A
k6	Load test	• Developer-friendly • Real-time metrics • Cloud execution	• CLI-first (learning)	Performance t
JMeter	Load test	• GUI • Distributed • Enterprise	• Slow GUI • Learning curve	Legacy/enterp
Gatling	Load test / simulation	• Scala DSL • High throughput • Report nice	• Learning curve	High-performance t
Trivy	Container/image scan	• Fast • Offline • Multi-format (image, SBOM)	• False positive	Container security
Snyk	Dependency/code/container	• Smart remediation • Auto PR • Cloud + CLI	• Freemium / paid	SaaS security
OWASP Dependency-Check	Dependency vulnerability	• Open source • Free • CVE database	• Less accurate than Snyk	Cost-sensitive
Semgrep	Code pattern / security	• Accurate • Custom rule • Offline	• Setup required • Slower	Code security audit

8. VÍ DỤ CI/CD PIPELINE: GitHub Actions

8.1 Scenario

Project Node.js + Express + PostgreSQL:

- Linting (ESLint)
- Unit test (Jest)
- Integration test (PostgreSQL + Testcontainers)
- Coverage check (>80%)
- Build Docker image
- Push to registry

8.2 GitHub Actions Workflow

File: `.github/workflows/ci.yml`

```
name: CI Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:14
        env:
          POSTGRES_USER: testuser
          POSTGRES_PASSWORD: testpass
          POSTGRES_DB: testdb
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
        ports:
          - 5432:5432
```

steps:

```
# Step 1: Checkout code
- name: Checkout code
  uses: actions/checkout@v3

# Step 2: Setup Node.js runtime
- name: Setup Node.js
  uses: actions/setup-node@v3
  with:
    node-version: '18'
    cache: 'npm' # Cache npm dependencies

# Step 3: Install dependencies
- name: Install dependencies
  run: npm ci # npm ci = clean install, more reliable than npm install

# Step 4: Run linter
- name: Run ESLint
  run: npm run lint

# Step 5: Run unit tests
- name: Run unit tests
  run: npm run test:unit

# Step 6: Run integration tests
- name: Run integration tests
  env:
    DB_HOST: localhost
    DB_PORT: 5432
    DB_USER: testuser
    DB_PASSWORD: testpass
    DB_NAME: testdb
  run: npm run test:integration

# Step 7: Generate coverage report
- name: Generate coverage report
  run: npm test -- --coverage

# Step 8: Check coverage threshold
- name: Check coverage
  run: npm run test:coverage:check

# Step 9: Upload coverage to Codecov
- name: Upload coverage to Codecov
  uses: codecov/codecov-action@v3
```

```

    with:
      files: ./coverage/lcov.info
      fail_ci_if_error: false

# Step 10: Build
- name: Build application
  run: npm run build

# Step 11: Build Docker image
- name: Build Docker image
  run: docker build -t myapp:${{ github.sha }} .

# Step 12: Push to Docker registry (optional, on main branch only)
- name: Push Docker image
  if: github.ref == 'refs/heads/main'
  run: |
    echo ${ secrets.DOCKER_PASSWORD } | docker login -u ${ secrets.DOCKER_USERNAME } --password-stdin
    docker tag myapp:${{ github.sha }} myapp:latest
    docker push myapp:${{ github.sha }}
    docker push myapp:latest

# Job 2: Security scanning (parallel)
security-scan:
  runs-on: ubuntu-latest

  steps:
    - name: Checkout code
      uses: actions/checkout@v3

    # Dependency vulnerability scan
    - name: Snyk scan
      uses: snyk/actions/node@master
      env:
        SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
      with:
        args: --severity-threshold=high

    # Container image scan
    - name: Trivy scan
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: myapp:latest
        format: 'sarif'
        output: 'trivy-results.sarif'

```

```
- name: Upload Trivy results
  uses: github/codeql-action/upload-sarif@v2
  with:
    sarif_file: 'trivy-results.sarif'
```

8.3 Giải Thích Từng Bước

Step	Mục Đích	Giải Thích	Tool
Checkout	Download code từ repo	Cần source code để build/test	actions/checkout
Setup Node.js	Cài Node runtime	Chỉ định version (18), cache npm	actions/setup-node
Install dependencies	npm install	npm ci an toàn hơn npm install (lock version)	npm
Lint	Kiểm tra code style	ESLint fail → stop pipeline	ESLint
Unit test	Test function riêng	Jest chạy test/*.test.js, report JSON	Jest
Integration test	Test module interaction	Connect database, run test	PostgreSQL + Jest
Coverage	Đo code coverage	Tạo report, check threshold >80%	Jest + coverage.py
Upload coverage	Track history	Codecov lưu trend, comment PR	Codecov
Build	Build application	npm run build → dist/ folder	webpack/esbuild
Docker build	Containerize app	Build image từ Dockerfile	Docker
Docker push	Push registry	Chỉ chạy trên main branch, push Docker Hub	Docker + secrets
Security scan	Phát hiện vulnerability	Snyk scan dependency, Trivy scan image	Snyk + Trivy

8.4 Status Badge

Add to README.md:

```
![CI Pipeline](https://github.com/user/repo/workflows/CI%20Pipeline/badge.svg)
[![codecov](https://codecov.io/gh/user/repo/branch/main/graph/badge.svg)]
(https://codecov.io/gh/user/repo)
```

9. CÁC VẤN ĐỀ THỰC TẾ KHI ÁP DỤNG CI/CD TESTING

9.1 Flaky Test

Định nghĩa:

Test chạy lần này pass, lần kia fail, mà code không thay đổi. Nguyên nhân từ test, không phải code.

Nguyên nhân:

- **Timing issue:** Test không chờ đủ element load, race condition
- **Random data:** Test dùng random số, UUID → output khác mỗi lần
- **External service:** Mock không được, call real service lỗi đôi khi
- **Order dependency:** Test A set data, test B dùng dữ liệu của A → fail nếu A chạy trước B
- **System resource:** Chạy khi CPU cao → test timeout
- **Database state:** Data cleanup không hết, test xem dữ liệu cũ

Hậu quả:

- Developer mất tin tưởng test ("This test fails randomly")
- Pipeline fail ngẫu nhiên → re-run phí thời gian
- Hard to debug: lỗi không reproduce locally

Cách khắc phục:

1. **Fix timing:** Dùng explicit wait, polling

```
// Bad: fixed timeout
await page.wait(1000);

// Good: wait for element
await page.waitForSelector('#user-list', { timeout: 5000 });
```

2. **Use fixed data:** Không random, dùng fixture

```
// Bad: random email
const email = `user-${Math.random()}@example.com`;

// Good: fixed email
const email = 'test-user@example.com';
```

3. **Mock external service:** Không call real service

```
// Bad: call real payment service
await paymentService.charge(...);
```

```
// Good: mock
paymentService.charge = jest.fn().mockResolvedValue({ success: true });
```

4. Test independence: Mỗi test clear data, setup độc lập

```
beforeEach(() => {
  // Clear database before mỗi test
  database.clearAllTables();
  // Setup fresh data
  seedTestData();
});
```

5. Retry flaky test: Nếu flaky rate thấp (~5%), set retry

```
// Jest: retry 3 times before fail
jest.retryTimes(3);
```

6. Log detail: Khi fail, log state để debug

```
try {
  // assertion
} catch (error) {
  console.log('Current page URL:', page.url());
  console.log('Page content:', await page.content());
  throw error;
}
```

9.2 Pipeline Chạy Quá Chậm

Vấn đề:

Pipeline cần 30 phút để chạy xong → feedback chậm → developer không dùng → lỗi không phát hiện sớm.

Nguyên nhân:

- **Quá nhiều test:** 10,000 test, mỗi 50ms = 500s = 8 phút
- **Test chạy tuần tự:** Chỉ 1 CPU core → build + test serial
- **Test I/O bound:** Network request, database query chậm
- **Build chậm:** Java compilation, webpack bundling lâu
- **Artifact push chậm:** Upload Docker image lớn

Hậu quả:

- Developer chờ lâu, feedback chậm
- Pipeline resource wasteful (VM chạy lâu)

- Bottleneck: merge bị block, deploy bị delay

Cách khắc phục:

1. **Parallelize test:** Chạy test song song trên nhiều worker

```
# GitHub Actions: parallelize across matrix
strategy:
  matrix:
    node: [18, 20]
    test-suite: [unit, integration, api]

# Jest: run parallel (default)
npm test -- --maxWorkers=4
```

2. **Split test:** PR pipeline = fast test, nightly = slow test

```
# PR: only unit test (3 min)
- if: github.event_name == 'pull_request'
  run: npm run test:unit

# Main: all test (15 min)
- if: github.ref == 'refs/heads/main'
  run: npm run test
```

3. **Cache:** Cache dependency, build output

```
- uses: actions/setup-node@v3
  with:
    cache: 'npm' # Cache node_modules

- uses: actions/cache@v3
  with:
    path: dist/
    key: build-${{ github.sha }} # Cache build output
```

4. **Fail fast:** Linter fail → stop, không chạy test

```
- run: npm run lint
- run: npm run test # Only run if lint pass
```

5. **Optimize test:** Remove redundant test, consolidate

```
// Bad: 3 test cùng check same function
test('add 2+3=5');
test('add 5+5=10');
test('add 10+20=30');

// Good: 1 parametrized test
```



```
test.each([
  [2, 3, 5],
  [5, 5, 10],
  [10, 20, 30]
])('add %i + %i = %i', (a, b, expected) => {
  expect(add(a, b)).toBe(expected);
});
```

6. **Light image:** Docker image nhỏ → push nhanh

```
# Bad: full OS
FROM ubuntu:20.04
RUN apt-get update && apt-get install node npm

# Good: minimal base
FROM node:18-alpine
```

9.3 Test Environment Khác Production

Vấn đề:

Test chạy pass trên CI, nhưng fail trên production vì environment khác.

Nguyên nhân:

- **OS khác:** CI dùng Linux, production Windows
- **Library version khác:** CI cài v1.0, production v1.5
- **Database khác:** CI dùng SQLite, production PostgreSQL
- **Config khác:** CI URL khác production
- **Resource khác:** CI VM nhỏ, production besar

Hậu quả:

- Test không phát hiện production issue
- False confidence: "test pass, phải OK"
- Production release lỗi, emergency fix

Cách khắc phục:

1. **Parity:** CI environment giống production

```
# CI: use PostgreSQL like production
services:
  postgres:
    image: postgres:14 # Same version as production
```

2. **Docker:** Use container để consistent

```
# Dockerfile production
FROM node:18-alpine
RUN npm ci --only=production

# CI: run same container
- run: docker build -t myapp .
- run: docker run myapp npm test
```

3. Infrastructure as Code: Terraform/CloudFormation describe environment

```
# Terraform: same DB instance config for test and prod
resource "aws_rds_instance" "test" {
  instance_class = "db.t3.micro"
  engine_version = "14.0"
}

resource "aws_rds_instance" "prod" {
  instance_class = "db.r5.large"
  engine_version = "14.0" # Same version
}
```

4. Staging environment: Test trên staging (giống production) sebelum production

```
# Deploy to staging first
- name: Deploy staging
  run: kubectl apply -f k8s-staging.yaml

- name: Smoke test staging
  run: npm run test:smoke -- --env=staging

# If pass, deploy to production
- name: Deploy production
  if: success()
  run: kubectl apply -f k8s-prod.yaml
```

5. Test dengan production-like data: Dùng real data shape (besar dataset, complex scenario)

9.4 Test Data Không Ổn Định

Vấn đề:

Test fail vì test data corrupt, incomplete, hoặc inconsistent.

Nguyên nhân:

- **Manual seeding:** Dev tạo data bằng tay (SQL script), dễ sai
- **No cleanup:** Test A tạo data, cleanup không hết → test B xem dữ liệu cũ

- **Shared database:** Multiple test dùng cùng database → race condition
- **Data dependency:** Test C dùng dữ liệu của test B → fail nếu B skip

Hậu quả:

- Test fail ngẫu nhiên
- Hard to debug: lỗi từ data, không phải code
- Pipeline flaky

Cách khắc phục:

1. **Factory method:** Tạo test data programmatically

```
function createTestUser(overrides = {}) {  
  return {  
    id: 1,  
    name: 'Test User',  
    email: 'test@example.com',  
    created_at: new Date(),  
    ... overrides  
  };  
}
```

2. **Builder pattern:** Flexible test data

```
new UserBuilder()  
  .withName('John')  
  .withEmail('john@example.com')  
  .withRole('admin')  
  .build();
```

3. **Fixture file:** JSON fixture reuse

```
// fixtures/users.json  
{  
  "admin": {  
    "id": 1,  
    "name": "Admin",  
    "role": "admin"  
  },  
  "user": {  
    "id": 2,  
    "name": "User",  
    "role": "user"  
  }  
}
```

4. Cleanup: Delete data sau mỗi test

```
afterEach(async () => {  
  await database.deleteAll('users');  
  await database.deleteAll('orders');  
});
```

5. Transaction rollback: Test chạy trong transaction, rollback sau

```
beforeEach(async () => {  
  await database.startTransaction();  
});  
  
afterEach(async () => {  
  await database.rollback(); // Restore original state  
});
```

6. Isolated database: Mỗi test dùng database instance riêng (container)

```
# Docker Compose: mỗi test dùng port khác  
postgres-test-1:  
  image: postgres:14  
  ports:  
    - "5432:5432"  
  
postgres-test-2:  
  image: postgres:14  
  ports:  
    - "5433:5433"
```

9.5 Secret / Config Bị Lộ Hoặc Cấu Hình Sai

Vấn đề:

API key, password hardcoded trong code → lộ secret, hoặc config sai → test call production.

Nguyên nhân:

- **Hardcode:** `const API_KEY = "sk-123456789"` trong source code
- **Config file:** `.env` được commit vào git
- **Wrong endpoint:** Test dùng `https://production-api.com` thay vì `https://staging-api.com`
- **No secret management:** Secret không được store an toàn

Hậu quả:

- Security breach: attacker dùng leaked API key

- Production damage: test delete production data
- Compliance issue: chứa PII (email, phone) trong test data

Cách khắc phục:

1. **Environment variable:** Secret ở environment, không code

```
// Good: read from env
const apiKey = process.env.API_KEY;

// Never hardcode
// const apiKey = "sk-123456";
```

2. **Secret management tool:** GitHub Secrets, AWS Secrets Manager

```
# GitHub Actions: store secret securely
- name: Deploy
  env:
    API_KEY: ${ secrets.API_KEY }
  run: deploy.sh
```

3. **Pre-commit hook:** Scan commit có secret trước khi push

```
# Install: npm install husky
# hook: scan cho pattern "password=", "api_key=", etc
```

4. **Test .env:** Create `.env.test` với dummy data

```
# .env.test
API_KEY=test-dummy-key-12345
API_URL=https://staging-api.example.com
DB_URL=postgres://test:test@localhost/testdb

# .gitignore: don't commit real .env
/.env
/.env.production
```

5. **Config validation:** Verify config trước run test

```
if (process.env.NODE_ENV === 'production') {
  if (process.env.API_URL.includes('staging')) {
    throw new Error('Wrong API URL: staging endpoint in production');
  }
}
```

6. **Rotation:** Regularly rotate secret

```
# Monthly: re-generate API key, update GitHub Secret
```

9.6 Report Khó Đọc

Vấn đề:

Test fail, nhưng report không rõ nguyên nhân, developer phải re-run để debug.

Nguyên nhân:

- **Generic assert:** `expect(result).toBe(null)` không rõ expected là gì
- **No screenshot:** E2E test fail, không screenshot → không biết UI sai gì
- **No log:** Không log state khi fail
- **Format xấu:** Report console dài, khó scroll
- **No link:** Report không link tới code, commit, PR

Hậu quả:

- Debug chậm: phải re-run test local, waste time
- Low engagement: developer skip test, trust giảm

Cách khắc phục:

1. **Clear assertion message:** Mô tả expected + actual

```
// Bad
expect(response.statusCode).toBe(200);

// Good
expect(response.statusCode).toBe(200,
  `Expected response to succeed, got ${response.statusCode}:
  ${response.body}`);
```

2. **Screenshot on failure:** E2E test fail → capture screenshot

```
afterEach(async (testInfo) => {
  if (testInfo.status !== 'passed') {
    await page.screenshot({ path: `failed-${Date.now()}.png` });
  }
});
```

3. **Structured log:** JSON log dễ parse

```
console.log(JSON.stringify({
  event: 'assertion_failed',
  test: 'createUser',
  expected: { id: 1, name: 'John' },
  actual: { id: undefined, name: 'John' },
```

```
timestamp: new Date()
});
```

4. HTML report: Nice UI report

```
# Allure report
npm test -- --reporter allure-reporter
allure serve
```

5. Link context: Report include code link, commit

```
# CI: attach metadata
- name: Publish test report
  if: always()
  uses: actions/upload-artifact@v3
  with:
    name: test-report-${{ github.sha }}
    path: coverage/
```

6. Failure summary: Highlight fail test

```
# Test output
FAIL src/user.test.js
  UserService
    ✓ should create user (45ms)
    ✗ should update user (32ms)
      Expected: { name: 'Updated' }
      Received: { name: 'John' }

SUMMARY: 1 passed, 1 failed
```

9.7 Coverage Cao Nhưng Test Kém Chất Lượng

Vấn đề:

Coverage 90%, nhưng bug vẫn ship. Test chỉ cover line chứ không cover logic.

Nguyên nhân:

- **Superficial test:** Test chạy function nhưng không assert output

```
// Bad: không assert
test('getUser') {
  userService.getUser(1); // Line covered, but no assert
}
```

- **Mock quá nhiều:** Test không test real logic, mock hết

```
// Bad: mock database
userService.database.find = jest.fn().mockResolvedValue(user);
```

```
// Vậy test gì? Test mock? Không test thực logic database interaction
```

- **No edge case:** Test happy path only

```
// Bad: chỉ test success
test('pay') {
  payment.process(100); // Happy path
  // Missing: payment fail, timeout, insufficient fund
}
```

- **No assertion logic:** Assert chỉ type, không value

```
// Bad: chỉ check type
expect(typeof result).toBe('number');

// Good: check value range
expect(result).toBeGreaterThan(0);
expect(result).toBeLessThan(100);
```

Hậu quả:

- False confidence: coverage cao, bug vẫn ship
- Maintenance overhead: test không giúp, lại cần maintain

Cách khắc phục:

1. **Coverage tool:** Measure line vs branch

```
npm test -- --coverage

# Report:
# - Line coverage: 90% (line executed)
# - Branch coverage: 60% (all if/else branch taken)
```

2. **Meaningful test:** Assert output, behavior

```
// Good: test behavior
test('discount 10% for order >100') {
  const discount = pricing.calculateDiscount(150);
  expect(discount).toBe(15); // 150 * 10%
}
```

3. **Edge case:** Test boundary, error

```
test('calculateDiscount', () => {
  expect(pricing.calculateDiscount(0)).toBe(0); // Boundary
  expect(pricing.calculateDiscount(99)).toBe(0); // Just below threshold
  expect(pricing.calculateDiscount(100)).toBe(10); // Just above threshold
  expect(pricing.calculateDiscount(1000)).toBe(100); // Large value
})
```



```
expect(() => pricing.calculateDiscount(-1)).toThrow(); // Error case
});
```

4. **Test mutation:** Mutation test tool (change code, test should fail)

```
# Stryker mutation testing
npm run stryker

# Killed 89%, Survived 11%
# → Some test didn't catch mutation (weak test)
```

5. **Code review:** Peer review test code, suggest improvement

9.8 Dependency / Third-Party Service Làm Test Fail

Vấn đề:

Test fail vì third-party service (payment, email, SMS) timeout hoặc down.

Nguyên nhân:

- **Real service call:** Test call thực payment gateway → fail nếu service down
- **Rate limit:** Service throttle request, test timeout
- **Network issue:** Latency cao, timeout
- **No retry:** Transient failure không retry

Hậu quả:

- Test flaky: fail randomly khi service unreliable
- Pipeline fail: không thể release, blocking
- False alarm: not code bug, service issue

Cách khắc phục:

1. **Mock service:** Unit/integration test mock, E2E test use staging

```
// Unit/integration: mock
paymentService.charge = jest.fn().mockResolvedValue({ success: true });

// E2E: use staging payment service (test account)
```

2. **Circuit breaker:** Stop calling service nếu sering fail

```
class PaymentService {
  async charge(amount) {
    if (this.failureCount > 5) {
      throw new Error('Service unavailable, circuit breaker open');
    }
  }
}
```

```

    try {
      return await externalPaymentGateway.charge(amount);
    } catch (error) {
      this.failureCount++;
      throw error;
    }
  }
}

```

3. **Retry with backoff:** Retry dengan exponential backoff

```

async function callWithRetry(fn, maxRetry = 3, delay = 100) {
  for (let i = 0; i < maxRetry; i++) {
    try {
      return await fn();
    } catch (error) {
      if (i === maxRetry - 1) throw error;
      await sleep(delay * Math.pow(2, i)); // Exponential backoff
    }
  }
}

```

4. **Timeout:** Set timeout, không chờ vô tận

```

const promise = paymentService.charge(100);
const timeoutPromise = new Promise((_, reject) =>
  setTimeout(() => reject(new Error('Timeout')), 5000)
);
return Promise.race([promise, timeoutPromise]);

```

5. **Test with staging/sandbox:** E2E test dùng sandbox account

```

# CI: use Stripe test API key
env:
  STRIPE_KEY: ${ secrets.STRIPE_TEST_KEY } # Test key, not production

```

6. **Monitoring:** Alert nếu test fail, investigate service

```

# If test fail, check service status
- name: Check service status
  if: failure()
  run: |
    curl https://payment-gateway-status.com
    # Alert team if down

```

10. BEST PRACTICES

10.1 Fast Feedback

Mục tiêu: Developer biết kết quả test nhanh nhất → sửa lỗi nhanh.

Cách thực hiện:

- PR pipeline chạy <10 phút (unit + lint)
- Parallelize test (4-8 worker)
- Cache dependency, build output
- Fail fast: linter fail → stop, không test tiếp

10.2 Fail Early

Mục tiêu: Phát hiện lỗi sớm nhất, chi phí fix thấp.

Cách thực hiện:

- Linter & format check trước test (syntax error sớm)
- Unit test trước integration test (isolate issue)
- Smoke test sau deploy (deployment error sớm)

10.3 Test Isolation

Mục tiêu: Mỗi test độc lập, không phụ thuộc test khác.

Cách thực hiện:

- beforeEach: setup fresh data
- afterEach: cleanup data, close connection
- Mock external service
- Không share state giữa test

10.4 Repeatability

Mục tiêu: Test chạy lần này lần kia kết quả giống nhau.

Cách thực hiện:

- Use fixed data, không random
- Mock time: `jest.useFakeTimers()`
- No flaky: explicit wait, retry
- Seed data consistent

10.5 Environment as Code

Mục tiêu: Environment reproducible, version-controlled.

Cách thực hiện:

- Docker Compose: infrastructure code
- Terraform: cloud resource code
- Configuration file: app config versioned
- No manual setup: script lại

10.6 Quality Gate

Mục tiêu: Enforce quality standard, không cho merge code kém chất lượng.

Cách thực hiện:

- Require: test pass, coverage >80%, lint pass
- Branch protection rule: PR require approval + check pass
- Automate: không dựa vào manual review (slow)

10.7 Parallelization

Mục tiêu: Giảm pipeline execution time.

Cách thực hiện:

- Parallel job: build + security scan cùng lúc
- Parallel test worker: Jest 4-8 worker
- Parallel E2E: run browser parallel (multiple user session)

10.8 Cache Dependency Hợp Lý

Mục tiêu: Tăng speed, giảm network I/O.

Cách thực hiện:

- Cache npm dependencies (node_modules)
- Cache build output (dist/, build/)
- Invalidate cache: khi package.json change
- Không cache test output (data thay đổi)

10.9 Không Chạy Mọi Loại Test Mọi Thời Điểm

Mục tiêu: Balance speed vs coverage.

Cách thực hiện:

- PR pipeline: unit + lint (3-5 min)
- Nightly: all test + E2E + load test (30-60 min)
- Release: all + security + manual test (strict)

Pipeline strategy:

On PR push:

- └─ Unit test (parallel)
- └─ Lint
- └─ Integration test
- └─ Coverage check

On schedule (nightly):

- └─ All test
- └─ E2E test
- └─ Performance test
- └─ Security deep scan

On release tag:

- └─ All test (strict)
- └─ Security scan (comprehensive)
- └─ Manual approval
- └─ Deploy production

10.10 PR Pipeline Nhanh, Nightly Deep, Release Strict

PR Pipeline (immediate feedback):

- Unit test
- Lint
- Basic integration test
- Duration: <10 min

Nightly Pipeline (comprehensive coverage):

- All test
- E2E test
- Performance test
- Security deep scan
- Duration: 1-2 hour

Release Pipeline (strict, manual approval):

- All test strict (no skip)
- Security scan comprehensive
- Manual approval required
- Deploy staging → test → manual check → production

10.11 Report / Log / Artifact Easy to Read

Mục tiêu: Developer dễ debug khi fail.

Cách thực hiện:

- Clear test name: `testLoginWithValidCredentialsShouldSucceed`
 - Detailed assertion message
 - Screenshot on failure (E2E)
 - Structured log (JSON)
 - HTML report (pretty UI)
 - Link tới code, commit, PR
-

11. GỢI Ý CẤU TRÚC SLIDE SEMINAR

Đề xuất **15-18 slide** cho presentation seminar:

Slide 1: Title Slide

- Tiêu đề: "CI/CD & Test-Harness Engineering"
- Tên tác giả, ngày
- Visualization: CI/CD pipeline diagram

Slide 2: Agenda

- Bullet: Khái niệm, Pipeline, Test Harness, Best Practice

Slide 3-4: Why CI/CD Matters

- Slide 3: Problem (manual test + late integration → lỗi)
- Slide 4: Solution (CI/CD → feedback nhanh, lỗi phát hiện sớm)
- Visualization: so sánh traditional vs CI/CD timeline

Slide 5: CI/CD Definition

- CI definition: frequent integration + auto test

- CD definition: delivery ready + auto deploy option
- Diagram: CI/CD flow

Slide 6: Pipeline Stages Overview

- Timeline: Trigger → Lint → Build → Test → Coverage → Security → Deploy
- Hình minh họa: pipeline block diagram

Slide 7-8: Test Pyramid

- Slide 7: Test pyramid (unit → integration → E2E)
- Slide 8: Loại test (unit, integration, API, E2E, smoke, load, security)
- Chi tiết từng loại

Slide 9: Test Harness Components

- Test runner, script, data, fixture, mock, stub, driver
- Diagram: test harness architecture

Slide 10: Test Harness Example

- Ví dụ: Node.js + PostgreSQL + Jest + Docker Compose
- Code snippet: test case

Slide 11: CI/CD Platforms & Tools

- Bảng so sánh: GitHub Actions, GitLab CI, Jenkins, CircleCI
- Tool groups: linter, test framework, coverage, etc.

Slide 12: Hands-on: GitHub Actions Example

- YAML workflow config
- Giải thích step-by-step
- Output result

Slide 13-14: Common Issues & Solutions

- Slide 13: Flaky test, slow pipeline
- Slide 14: Wrong environment, flaky data, secret leak
- Fix strategies

Slide 15: Best Practices

- Fast feedback, fail early, test isolation
- Repeatability, quality gate, parallelize

Slide 16: Common Mistakes

- All test every time (slow)
- No mock (flaky)
- Coverage obsession
- Test after code freeze (late)

Slide 17: Practical Tips

- Start small: lint + unit test
- Gradually add: integration → E2E
- Monitor: coverage trend, test speed, flaky rate
- CI/CD là investment, không burden

Slide 18: Conclusion

- CI/CD không thay thế tester, mà empower tester
 - Test harness = foundation
 - Continuous improvement mindset
 - Q&A
-

12. KẾT LUẬN

12.1 Tóm Tắt Vai Trò CI/CD trong Kiểm Thử Phần Mềm

CI/CD là quy trình tự động hóa không thể thiếu trong kiểm thử phần mềm hiện đại:

1. **Tích hợp liên tục (CI):** Code tích hợp vào repository thường xuyên (mỗi ngày), mỗi lần tích hợp được kiểm tra tự động bằng test, lint, build.
2. **Phân phối/triển khai liên tục (CD):** Sau khi pass CI, code sẵn sàng để deploy (delivery) hoặc tự động deploy (deployment).
3. **Feedback nhanh:** Developer nhận phản hồi trong vài phút, không phải chờ ngày hôm sau.
4. **Phát hiện lỗi sớm:** Lỗi được catch ở pipeline, không bao giờ lên production → chi phí fix thấp.
5. **Release an toàn:** Có thể deploy nhiều lần một ngày vì test kiểm chứng chất lượng.

12.2 Tóm Tắt Vai Trò Test Harness

Test Harness là infrastructure để chạy test một cách tự động, lặp lại, đáng tin cậy:

1. **Tích hợp các thành phần:** Test runner, framework, data, fixture, mock, environment, reporter.
2. **Consistency:** Mỗi lần test chạy setup giống nhau → result reproducible, không flaky.
3. **Scalability:** Có thể chạy hàng ngàn test tự động mà không cần tester chạy thủ công.
4. **Isolation:** Mock external service → test độc lập, không phụ thuộc bên ngoài.
5. **Debugging:** Log, screenshot, trace → dễ debug khi test fail.

12.3 Thông Điệp Chính

CI/CD và Test Harness Engineering không thay thế tester, mà giúp tester:

1. **Tự động hóa task lặp lại:** Tester không cần chạy manual test hàng trăm lần, mà hệ thống tự động chạy.
2. **Phát hiện lỗi sớm:** Lỗi phát hiện ở PR stage, không đợi QA phase → tiết kiệm thời gian, chi phí.
3. **Focus vào high-value test:** Tester focus vào exploratory test, usability test, security audit → công việc ý nghĩa hơn.
4. **Release an toàn hơn:** Pipeline kiểm chứng quality → confidence cao khi release.
5. **Continuous improvement:** Trend test coverage, pipeline speed, bug tìm được → data-driven improvement.

Mục tiêu cuối cùng: Build process kiểm thử mạnh mẽ, tự động, đáng tin cậy → phần mềm chất lượng cao, release nhanh, an toàn.

TÀI LIỆU THAM KHẢO VÀ TÀI NGUYÊN

Official Documentation

- GitHub Actions: <https://docs.github.com/en/actions>
- GitLab CI/CD: <https://docs.gitlab.com/ee/ci/>
- Jenkins: <https://www.jenkins.io/doc/>
- SonarQube: <https://docs.sonarqube.org/>
- Docker: <https://docs.docker.com/>
- Testcontainers: <https://www.testcontainers.org/>

Tool Documentation

- Jest: <https://jestjs.io/>

- pytest: <https://pytest.org/>
- JUnit: <https://junit.org/junit5/>
- Playwright: <https://playwright.dev/>
- Cypress: <https://cypress.io/>

Best Practices & Articles

- Continuous Integration: <https://martinfowler.com/articles/continuousIntegration.html>
 - Test Pyramid: <https://martinfowler.com/bliki/TestPyramid.html>
 - Flaky Tests: <https://martinfowler.com/articles/testing-strategies.html>
-

Tài liệu này được biên soạn để chuẩn bị seminar "CI/CD & Test-Harness Engineering"

Phiên bản: v1.0 | Ngày tạo: 2026

```
Dataview (inline field '=='): Error:
```

```
-- PARSING FAILED -----  
-
```

```
> 1 | ==  
    | ^
```

Expected one of the following:

```
('(', 'null', boolean, date, duration, file link, list ('[1, 2,  
3]'), negated field, number, object ('{ a: 1, b: 2 }'), string,  
variable
```